01 - Product Requirements Document

Visor DOM-to-Agent Context Compiler
DOC-01 | 0.1 requirements pack | Generated 2026-06-14

This PRD defines the product problem, users, use cases, priorities, MVP feature set, metrics, and risk assumptions for Visor.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-01
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Product problem

AI agents need structured context, but web pages are built for humans and browsers. Raw DOM contains repeated navigation, hidden nodes, scripts, style noise, cookie banners, sidebars, irrelevant recommendations, and prompt-injection content. Visor compiles this messy input into a smaller, safer, structured context package.

2. Product thesis

The highest-value primitive is not a chatbot popup. The highest-value primitive is a reliable page compiler that can produce deterministic, inspectable, schema-valid context for any downstream agent.

3. Target personas

Persona	Need	Pain today	Visor value
Agent developer	Give a browser agent accurate page state.	Raw HTML is too noisy and unsafe.	Structured AgentContext with actions and token budget.
Researcher/student	Summarize and extract knowledge from pages.	Manual copy-paste loses structure.	Clean content, headings, tables, source URL.
RAG builder	Chunk docs and articles for retrieval.	Generic crawlers miss UI and context.	RAG mode with block metadata.
Automation builder	Identify forms/buttons/links.	Selectors and labels are messy.	Action mode with labels and selector hints.
Security-conscious user	Avoid leaking private page data.	Extensions often over-collect data.	Local-first preview, redaction, domain warnings.

4. Core jobs-to-be-done

- When I am viewing a page, I want to convert it into structured context so an AI agent can reason over it.
- When a page has forms or buttons, I want the agent to know what actions are possible without guessing.
- When a page contains private data, I want warnings and redaction before anything leaves my browser.
- When a page is long, I want a compressed version that fits within a token budget.
- When context looks wrong, I want debug visibility into what was kept, removed, and scored.

5. User stories

ID	Requirement	Priority	Acceptance signal
US-001	As a user, I can click Compile Page and receive structured context for the active tab.	P0	Popup shows schema-valid JSON preview.
US-002	As a user, I can choose compact, detailed, action, RAG, or debug output.	P0	Mode selector changes output content and fields.

ID	Requirement	Priority	Acceptance signal
US-003	As an agent builder, I can copy context as JSON or Markdown.	P0	Clipboard output matches selected format.
US-004	As a privacy-conscious user, I can enable strict redaction.	P0	Emails, phones, tokens, and password fields are masked.
US-005	As a developer, I can define site-specific selectors to preserve or ignore.	P1	Site profile changes extraction result for that domain.
US-006	As a tester, I can view debug notes for removed noise.	P1	Debug view lists kept/removed blocks and reasons.
US-007	As a future cloud user, I can opt in to saving context history.	P2	Cloud mode requires explicit consent and clear retention policy.

6. Feature prioritization

Feature	MVP priority	Why
DOM extraction	P0	Core product value; everything depends on it.
AgentContext schema	P0	Provides stable output contract for agents.
Privacy redaction	P0	Required because extension reads page content.
Preview/export	P0	User must inspect before sharing.
Site profiles	P1	Needed for quality on recurring websites.
RAG chunks	P1	Useful for docs and long pages.
Cloud accounts	P2	Not needed to prove the compiler.
Marketplace	P3	Explicitly future only.

7. Success metrics

- Extraction success rate across test pages: 95% without crashes.
- Average compile time on normal article page: under 2 seconds.
- Raw text reduction: at least 40% token reduction on noisy pages while preserving main content.
- Schema validity: 100% of generated outputs pass AgentContext validation.
- Privacy obvious-hit coverage: emails, phones, JWT-like tokens, password fields detected in fixture tests.
- User trust metric: preview always displays privacy warning status before export.

8. Product risks

Risk	Impact	Mitigation
Over-permissioned extension	User distrust and platform rejection.	Use minimal permissions and explain each permission.
Bad extraction quality	Product feels useless.	Use fixture pages, site profiles, debug mode, and scoring tests.
Private data leakage	Severe trust and legal risk.	Local-first, preview, redaction, no default backend.
Prompt injection from webpage	Agent compromise downstream.	Mark page text as untrusted and isolate it from instructions.
Large-page performance	Browser freeze.	Chunking, node limits, async processing, graceful fallback.